```
    public IEnumerable<ValidationResult> Validate(                ❸
        ValidationContext validationContext)                      ❸
    {
        if(string.IsNullOrEmpty(Email)                            ❹
            && string.IsNullOrEmpty(PhoneNumber))                 ❹
        {
            yield return new ValidationResult(                    ❺
                "You must provide an Email or a PhoneNumber",     ❺
                New[] { nameof(Email), nameof(PhoneNumber) });    ❺
        }
    }
}
```

❶ Implements the IValidatableObject interface

❷ The DataAnnotation attributes continue to validate basic format requirements.

❸ Validate is the only function to implement in IValidatableObject.

❹ Checks whether the object is valid . . .

❺ . . . and if not, returns a result describing the error

`IValidatableObject` helps cover some of the cases that attributes alone can't handle, but it's not always the best option. The `Validate` function doesn't give easy access to your app's services, and the function executes only if all the `DataAnnotation` attribute conditions are met.

**TIP** `DataAnnotations` are good for input validation of properties in isolation but not so good for validating complex business rules. You'll most likely need to perform this validation outside the `DataAnnotations` framework.

Alternatively, if you're not a fan of the `DataAnnotation` attribute-based-plus-`IValidatableObject` approach, you could use the popular FluentValidation library (https://github.com/JeremySkinner/FluentValidation) in your minimal APIs instead. Minimal APIs are completely flexible, so you can use whichever approach you prefer.